Workshop – Week 3 – Some selected solutions

1. What is the hexadecimal of the bit pattern for 32-bit storage of the floating point numbers:

a) 100.875 **b)** -0.75

Solution (a):
Convert the integer part of the number to binary: 100 = 1100100
Convert the fraction to binary: 0.875 = 875/1000 = 7/8
The columns to the right of the point are: 2^{-1}, 2^{-2}, 2^{-3}, etc, i.e. ½, ¼, 1/8, etc
0.875 = 7/8 = ½ + ¼ + 1/8 = 0.111 (in binary)

So we have converted 100.875 to the binary number 1100100.111
Convert the binary number to normal form: 1100100.111 = 1.100100111×2^6

Now break the number into sign bit, exponent (8 bits) and fraction (23 bits)

Sign bit = 0 (a positive number)

Exponent = 6 in excess 127 notation = 6 + 127 = 133 = 10000101

Fraction in 23 bits = 100 1001 1100 0000 0000 0000

Final bit pattern (32 bits) = 01000010110010011100000000000000 and hexadecimal is 42C9C000

Solution (b):
You do the working...

Final bit pattern (32 bits) = 10111111010000000000000000000000 and hexadecimal is BF400000

2. What floating point numbers are stored (32 bits) as the patterns with hexadecimal:

a) 42590000 **b)** BF900000

Solution (a):
Convert hexadecimal to binary: 42590000 = 0100 0010 0101 1001 0000 0000 0000 0000
Split off the various sections:

Sign bit = 0 (a positive number)

Exponent = 8 bits = 10000100 = 128 + 4 = 132 (decimal)
Convert exponent from excess 127 notation: 132 – 127 = 5 (exponent is 5 in decimal)

Fraction = 23 bits = 1011001

So the number in binary is 1.1011001×2^5 = 110110.01

Convert the two parts of the number to decimal:
integer part = 110110 = 32 + 16 + 4 + 2 = 54
fraction = 0.01 = ¼ = 0.25

Final decimal number is 54.25

Solution (b):
You do the working...

Final decimal number is -1.125

3. Working in **4 bits**, give answers in binary, hexadecimal and decimal to:

a) 6 & 3 **b)** 9 ^ 5

Solution (a):

```
6 = 0110
3 = 0011 AND
   ----------
    0010
```

The answer is 0010 (binary), 2 (in hexadecimal) and 2 (in decimal)

Solution (b):
You do the working...

The answer is 1100 (binary), C (in hexadecimal) and 12 (in decimal)

4. Working in **8 bits**, give answers in binary, hexadecimal and decimal to:

a) C3 | 5F **b)** DE << 3

Solution (a):

```
C3 =  1100 0011
5F =  0101 1111 OR
     ------------------
      1101 1111
```

The answer is 11011111 (binary), DF (in hexadecimal) and 223 (in decimal)

Alternative solution to (a): (can also be used in Question 3 above)

Run the program:

```
int a, b, c;

int main() {
  a = 0xC3;
  b = 0x5F;
  c = a | b;
  cout << c << endl;
}
```

Solution (b):

DE = 1101 1110
shift 3 bits to the right = 1111 0000 (3 bits have fallen off the left-hand side)
The answer is 11110000 (binary), F0 (in hexadecimal) and 240 (in decimal)

Another way to check your solution to (b):

DE = 16 * 13 + 1 * 14 = 222 (decimal)
shifting 3 bits to the left is the same as multiplying by 8 (2^3) and 222 x 8 = 1,776 (decimal)
1,776 convert to hexadecimal = 1,776 / 16 = 111 rem 0, 111 / 16 = 6 rem 15, 6 / 16 = 0 rem 6
Write the remainders backwards and we get 1,776 (decimal) = 6F0 (hexadecimal)
6F0 (hexadecimal) = 0110 1111 0000 (binary)
But we are working in 8 bits so keep only the 8 bits on the right-hand side = 11110000
Note that we get the same result as above.

6. Explain the difference in the results when using the following code:

```
c = 0xA4;
c = c >> 2;
cout << "c is " << (int) c << endl;
```

a) define `c` as a `char`

b) define `c` as an `unsigned char`

Solution:

An unsigned char means 8 bits are available for the calculation and the calculation proceeds as usual. A char means only 7 bits are available and when the highest bit is shifted, it goes into the next column which is actually the sign bit. And so we get a negative result.

Interesting note for programmers:

If a program is unexpectedly producing negative results, it could mean that your numbers are too big for the allocated variable.

7. `c` is an `unsigned char`. Write code that uses bitwise operations to find the value in:

a) the least significant bit (the bit on the extreme right-hand side of the number)

b) the most significant bit (the bit on the extreme left-hand side of the number)

c) the 3 bits on the right (the low end of the number)

d) the 3 bits on the left (the top end of the number)

Solution (b):

We are working with 8 bits (an unsigned char).
Set up a mask of 1000 0000 (80 in hexadecimal) – i.e. place a 1 in the most significant bit
Do an AND operation with the mask and the number you want to test
The AND causes the zeros in the mask to make any bits (except the most significant bit) = zero.

```
#include <iostream>
using namespace std;

unsigned char c, mask, result;

int main() {
  c = 0x9D;     // this is the number you are testing
  mask = 0x80;  // 1000 0000
  result = c & mask;
  if (result == 0) {
    cout << "The most significant bit is 0\n";
  } else {
    cout << "The most significant bit is 1\n";
  }
}
```

Some test data is: 5 (m.s.b will be zero) and 1001 1101 = 9D (m.s.b. will be one)